

多項式・形式的べき級数 線形代数 線形漸化的数列 C-recursive Fiduccia のアルゴリズム Bostan-Mori のアルゴリズム

線形漸化的数列の第 K 項

1. 概要

単位的可換環 R 上の数列 $A = (A_0, A_1, A_2, \dots)$ が、定数 $c_1, c_2, \dots, c_d$ について

$$A_i = c_1 A_{i-1} + c_2 A_{i-2} + \dots + c_d A_{i-d} \quad (i \geq d)$$

を満たすとします。このような数列を**線形漸化的数列**といい、次の講座でその性質を詳しく整理しました。

- 線形漸化的数列

本講座ではその続きとして、線形漸化式と初期値 $A_0, A_1, \dots, A_{d-1}$ が与えられたときに、数列の第 K 項 A_K を求める問題をより詳しく扱います。

この問題を解く方法のひとつは行列累乗を用いるもので、単純な方法では計算量が $O(d^3 \log K)$ 時間となります。本講座では、多項式を用いてより高速に第 K 項を求める方法を説明します。より具体的には、次数 d 程度の多項式積の計算量を $M(d)$ としたとき、 $O(M(d) \log K)$ 時間で A_K を求める方法として、Fiduccia のアルゴリズムと Bostan-Mori のアルゴリズムを扱います。

2. 前提知識

以下の講座を理解していることを前提とします。

- 線形漸化的数列

また、多項式の乗算・剰余算を扱います。高速な実装では FFT を用いた多項式乗算が必要になりますが、アルゴリズムの考え方を理解するだけであれば、素朴な多項式演算を考えれば十

分です.

3. 問題設定

本講座で扱うのは次の問題です.

【問題 1】

数列 $A = (A_0, A_1, A_2, \dots)$ が, 定数係数線形漸化式

$$A_i = c_1 A_{i-1} + c_2 A_{i-2} + \dots + c_d A_{i-d} \quad (i \geq d)$$

を満たしているとします. 漸化式の係数 $c_1, c_2, \dots, c_d$, 初期値 $A_0, A_1, \dots, A_{d-1}$ および非負整数 K が与えられたとき, 数列 A の第 K 項 A_K を求めてください.

ただし, 次の約束をします.

- 単位的可換環 R をひとつ固定し, 数列の要素や漸化式の係数はすべて R の元とします.
- R の環演算 (加算・減算・乗算) は $O(1)$ 時間で行えるものとします.

代数の用語に不慣れな場合, $R = \mathbb{Z}/n\mathbb{Z}$ または $\mathbb{F}_p$ と考えてください.

また, 計算量解析において, 次数 d の多項式積を計算する時間を $M(d)$ と書きます. 例えば素朴な多項式乗算では

$$M(d) = O(d^2)$$

です. 競技プログラミングにおいて頻出する R ($\mathbb{Z}/n\mathbb{Z}$, $\mathbb{F}_p$, $\mathbb{C}$ など) では, 適切な計算量モデルのもとで, $M(d) = O(d \log d)$ と考えることができます.

以降では, 第 K 項 A_K を $O(M(d) \log K)$ 時間で求める方法を説明します.

4. Fiduccia のアルゴリズム

本節では, 線形漸化的数列の第 K 項を求める古典的な方法である Fiduccia のアルゴリズムを説明します.

4.1. 線形漸化式と多項式剰余

数列 A が、定数係数線形漸化式

$$A_i = c_1 A_{i-1} + c_2 A_{i-2} + \cdots + c_d A_{i-d} \quad (i \geq d)$$

を満たすとします。この式をそのまま「左辺を右辺に置き換える操作が行える」と解釈すると、 A_i をより添字が小さい項の線形結合として表すことができます。このことを繰り返すと、 A_i を $A_0, A_1, \dots, A_{d-1}$ の線形結合として表すことができます。

具体例

例えば A が

$$A_i = 2A_{i-1} + 3A_{i-2} \quad (i \geq 2)$$

を満たすとします。 A_4 より添字が小さい項の線形結合で表してみましょう。まず漸化式で $i = 4$ とした式から

$$A_4 = 2A_3 + 3A_2$$

となります。次に漸化式で $i = 3$ とした式 $A_3 = 2A_2 + 3A_1$ を用いると、

$$A_4 = 2(2A_2 + 3A_1) + 3A_2 = 7A_2 + 6A_1$$

となります。さらに漸化式で $i = 2$ とした式 $A_2 = 2A_1 + 3A_0$ を用いると、

$$A_4 = 7(2A_1 + 3A_0) + 6A_1 = 20A_1 + 21A_0$$

となります。これで A_4 が A_0, A_1 の線形結合として表すことができました。

このように A_i を、 $A_0, A_1, \dots, A_{d-1}$ の線形結合で表すことができれば、第 i 項を初期値から求めることができます。Fiduccia のアルゴリズムでは、この線形結合の係数を多項式剰余として求めます。

多項式剰余との比較

多項式 $f(x), g(x), \Gamma(x)$ について、 $f(x) - g(x) = \Gamma(x)h(x)$ であるような多項式 $h(x)$ が存在するときに $f(x) \equiv g(x) \pmod{\Gamma(x)}$ と表すことにします。上の具体例の計算過程は、次のような多項式剰余を求める計算過程と類似していることが分かります。

$$\Gamma(x) = x^2 - (2x + 3)$$

とします。 x^4 を $\Gamma(x)$ で割った余りは次のように求められます。

$$x^2 \equiv 2x + 3 \pmod{\Gamma(x)} \quad (*)$$

であることに注意します.

まず $(*)$ の x^2 倍から

$$x^4 \equiv 2x^3 + 3x^2 \pmod{\Gamma(x)}$$

となります. 次に $(*)$ の x 倍から分かる式 $x^3 \equiv 2x^2 + 3x \pmod{\Gamma(x)}$ を用いると,

$$x^4 \equiv 2(2x^2 + 3x) + 3x^2 = 7x^2 + 6x \pmod{\Gamma(x)}$$

となります. さらに $(*)$ を用いると,

$$x^4 \equiv 7(2x + 3) + 6x \equiv 20x + 21 \pmod{\Gamma(x)}$$

となります. これで $x^4 \equiv r_0 + r_1x \pmod{\Gamma(x)}$ を満たす r_0, r_1 (言い換えれば, x^4 の $\Gamma(x)$ による剰余) を求めることができました.

定理

上の具体例で見た通り,

- 線形漸化的数列の A_i を $A_0, A_1, \dots, A_{d-1}$ の線形結合で表す問題
- x^i の $\Gamma(x)$ による剰余 $r_0 + r_1x + \dots + r_{d-1}x^{d-1}$ を求める問題

はほとんど同じ計算により解くことができます. ここから以下の定理が得られます.

【定義 2】(特性多項式)

定数係数線形漸化式

$$A_i = c_1 A_{i-1} + c_2 A_{i-2} + \dots + c_d A_{i-d} \quad (i \geq d)$$

に対して, 多項式

$$\Gamma(x) = x^d - c_1 x^{d-1} - c_2 x^{d-2} - \dots - c_d$$

を**特性多項式** (characteristic polynomial) という.

【定理 3】

数列 $A = (A_0, A_1, A_2, \dots)$ を位数 d の定数係数線形漸化式

$$A_i = c_1 A_{i-1} + c_2 A_{i-2} + \dots + c_d A_{i-d} \quad (i \geq d)$$

を満たす線形漸化的数列とし,

$$\Gamma(x) = x^d - c_1 x^{d-1} - c_2 x^{d-2} - \dots - c_d$$

をその特性多項式とする. 非負整数 i に対して x^i の $\Gamma(x)$ による剰余を

$$r_0 + r_1 x + \dots + r_{d-1} x^{d-1}$$

とすると,

$$A_i = r_0 A_0 + r_1 A_1 + \dots + r_{d-1} A_{d-1}$$

が成り立つ.

上の具体例で見たように, この定理は, $x^d \equiv c_1 x^{d-1} + \dots + c_d \pmod{\Gamma(x)}$ という置き換えと, $A_i = c_1 A_{i-1} + \dots + c_d A_{i-d}$ という置き換えが同じ係数を持つことから従います. ■

【問題 4】

数列 A を漸化式

$$A_i = A_{i-1} + A_{i-2} + A_{i-3} \quad (i \geq 3)$$

および初期値 $A_0 = 3, A_1 = 2, A_2 = 1$ で定まる数列とします.

1. 漸化式を用いて A_3, A_4, A_5 を求めてください.
2. A_5 を A_0, A_1, A_2 の線形結合として表し, そこから A_5 を求めてください.

▶ 解答

4.2. Fiduccia のアルゴリズム

前節の定理より、第 K 項 A_K を求めるには、 x^K の特性多項式 $\Gamma(x)$ による剰余を求めればよいことが分かりました。

$x^K \bmod \Gamma(x)$ は、繰り返し二乗法により計算できます。これを用いると、次の擬似コードによって A_K を求めることができます。

```
function Fiduccia(A, Gamma(x), K):
    # A[0], A[1], ..., A[d-1]: initial values
    # Gamma(x): characteristic polynomial of degree d
    # returns A_K

    function ModPow(K):
        # returns x^K mod Gamma(x)

        if K < d:
            return x^K

        R(x) := ModPow(floor(K / 2))
        R(x) := R(x)^2 mod Gamma(x)

        if K is odd:
            R(x) := x R(x) mod Gamma(x)

        return R(x)

    R(x) := ModPow(K)
    ans := 0
    for i = 0, 1, ..., d - 1:
        ans := ans + ([x^i]R(x)) * A[i]
    return ans
```

ここで、次数 d 未満の多項式どうしの積を $\Gamma(x)$ で割った余りを求める操作は、素朴な方法で $O(d^2)$ 時間で行えます。また本講座では解説を省略しますが、畳み込みと形式的べき級数逆元を用いて $O(M(d))$ 時間で行えることが知られています。

したがって、高速な多項式剰余算を用いることで、Fiduccia のアルゴリズムにより、線形漸化的数列の第 K 項 A_K を $O(M(d) \log K)$ 時間で求めることができます。

▶ 2 種の繰り返し二乗法による違い

4.3. 実装例

Typical DP Contest「T - フィボナッチ」への提出です。

- <https://atcoder.jp/contests/tdpc/submissions/76155973>

この実装では、多項式乗算・剰余算を素朴な方法により $O(d^2)$ 時間で計算しています。漸化式の位数を d 、求める添字を K とすると、解法全体の計算量は $O(d^2 \log K)$ 時間です（問題文の記号では、 $O(K^2 \log N)$ 時間です）。

5. Bostan-Mori のアルゴリズム

本節では、線形漸化的数列の第 K 項を求めるもうひとつの方法として、Bostan-Mori のアルゴリズムを解説します。

5.1. 線形漸化式と形式的べき級数（復習）

線形漸化的数列 で解説したように、線形漸化的数列 $A = (A_0, A_1, A_2, \dots)$ の母関数

$$A(x) = A_0 + A_1x + A_2x^2 + \dots$$

は有理式で表すことができます。数列 A が定数係数線形漸化式

$$A_i = c_1 A_{i-1} + c_2 A_{i-2} + \dots + c_d A_{i-d} \quad (i \geq d)$$

を満たすとします。このとき、母関数 $A(x)$ は $A(x) = \frac{P(x)}{Q(x)}$ と表せます。ここで

$$\begin{aligned} Q(x) &= 1 - c_1x - c_2x^2 - \dots - c_dx^d, \\ P(x) &= A(x)Q(x) \bmod x^d \end{aligned}$$

です。第 K 項 A_K は $A_K = [x^K] \frac{P(x)}{Q(x)}$ と表せます。Bostan-Mori のアルゴリズムは、この有理式の第 K 係数を高速に求めるアルゴリズムです。

なお $Q(x)$ は定数係数線形漸化式の係数を並べてできる多項式ですが、特性多項式とは係数の並び順が逆順であることに注意してください。

5.2. 形式的べき級数の偶数次部分，奇数次部分

形式的べき級数

$$F(x) = \sum_{i=0}^{\infty} F_i x^i = F_0 + F_1 x + F_2 x^2 + \cdots$$

に対して，その偶数次部分，奇数次部分を

$$F_e(x) = \sum_{i=0}^{\infty} F_{2i} x^i = F_0 + F_2 x + F_4 x^2 + \cdots,$$
$$F_o(x) = \sum_{i=0}^{\infty} F_{2i+1} x^i = F_1 + F_3 x + F_5 x^2 + \cdots.$$

により定めます。 $F(x) = F_e(x^2) + xF_o(x^2)$ が成り立ちます。

Bostan-Mori のアルゴリズムでは，次の補題を用います。

【補題 5】

$Q(x)$ を形式的べき級数とすると， $Q(x)Q(-x)$ の奇数次部分は 0 である。つまりある形式的べき級数 $V(x)$ が存在して，

$$Q(x)Q(-x) = V(x^2)$$

が成り立つ。

非負整数 i に対して， $[x^i]Q(x)$ を Q_i と書くことにします。非負整数 k に対して

$$[x^k]Q(x)Q(-x) = \sum_{i+j=k} ([x^i]Q(x)) \cdot ([x^j]Q(-x)) = \sum_{i+j=k} Q_i \cdot (-1)^j Q_j$$

が成り立ちます。さらに k が奇数であるとき， $i+j=k$ を満たす非負整数の組 (i, j) は次のようなものです。

- $a + b = k$ を満たす偶数 a , 奇数 b に対して $(i, j) = (a, b)$.
- $a + b = k$ を満たす偶数 a , 奇数 b に対して $(i, j) = (b, a)$.

同じ (a, b) に対するこれら 2 通りの (i, j) の寄与をまとめると,

$$Q_a \cdot (-1)^b Q_b + Q_b \cdot (-1)^a Q_a = (-Q_a Q_b) + Q_b Q_a = 0$$

となるので, $[x^k] Q(x) Q(-x) = 0$ が成り立ちます. ■

▶ 証明方法に関する補足

5.3. Bostan-Mori のアルゴリズム

定数係数線形漸化式を満たす数列の第 K 項を求める問題は,

- $d - 1$ 次以下の多項式 $P(x)$
- d 次以下の多項式 $Q(x)$ (ただし $[x^0] Q(x) = 1$)

に対して $[x^K] \frac{P(x)}{Q(x)}$ を求める問題に帰着できました. Bostan-Mori のアルゴリズムはこの問題を, 次のように再帰的に解くものです.

まず,

$$\frac{P(x)}{Q(x)} = \frac{P(x) Q(-x)}{Q(x) Q(-x)}$$

と変形します. 【補題 5】より, 分母はある d 次以下の多項式 $V(x)$ により $V(x^2)$ と書けます. 分子を $U(x) = P(x) Q(-x)$ とおきます. すると

$$\frac{P(x)}{Q(x)} = \frac{U_e(x^2)}{V(x^2)} + x \frac{U_o(x^2)}{V(x^2)}$$

が成り立ちます. この式は, $\frac{P(x)}{Q(x)}$ の偶数次部分, 奇数次部分への分解に対応します. つまり, $\frac{P(x)}{Q(x)}$ の偶数次部分は $\frac{U_e(x)}{V(x)}$, 奇数次部分は $\frac{U_o(x)}{V(x)}$ となります.

この式から, K の偶奇に応じて次が分かります.

$$\begin{aligned} [x^K] \frac{P(x)}{Q(x)} &= [x^{K/2}] \frac{U_e(x)}{V(x)} \quad (K \equiv 0 \pmod{2}), \\ [x^K] \frac{P(x)}{Q(x)} &= [x^{(K-1)/2}] \frac{U_o(x)}{V(x)} \quad (K \equiv 1 \pmod{2}). \end{aligned}$$

ここで右辺の分子に現れる多項式 $U_e(x)$, $U_o(x)$ は $d-1$ 次以下の多項式です. また右辺の分母に現れる多項式 $V(x)$ は d 次以下の多項式で, $[x^0] V(x) = 1$ を満たします. これで問題の形を保ったまま, 求める係数の添字 K を半分以下にすることができました.

この操作を繰り返すと, 最終的には $K = 0$ の場合, つまり $[x^0] \frac{P(x)}{Q(x)}$ を求める問題に帰着されますが, $[x^0] Q(x) = 1$ よりこの問題の答えは $[x^0] P(x)$ です.

まとめると, 以下の擬似コードにより A_K を求めることができます.

```
function Bostan_Mori(P(x), Q(x), K):
    # returns A_K = [x^K]P(x)/Q(x)
    # assume [x^0]Q(x)=1

    while K > 0:
        U(x) := P(x) Q(-x)
        W(x) := Q(x) Q(-x)
        if K is even:
            P(x) := U_e(x)
        else:
            P(x) := U_o(x)
        Q(x) := W_e(x)
        K := floor(K / 2)

    return [x^0]P(x)
```

これが Bostan-Mori のアルゴリズムです.

イテレーションごとに多項式乗算 $P(x)Q(-x)$, $Q(x)Q(-x)$ を計算するため, 計算量は $O(M(d) \log K)$ 時間となります.

【問題 6】

数列 $F = (F_0, F_1, F_2, \dots) = (0, 1, 1, 2, 3, 5, 8, 13, 21, \dots)$ を Fibonacci 数列とすると, F の母関数は $F(x) = \frac{x}{1-x-x^2}$ となるのでした.

1. $F_0(x) = \frac{U(x)}{V(x)}$ を満たす多項式 $U(x), V(x)$ を求めてください.
2. 上で求めた $U(x), V(x)$ について $\frac{U(x)}{V(x)}$ について低次の係数を計算することで,
 $\frac{U(x)}{V(x)} = F_1 + F_3x + F_5x^2 + F_7x^3 + \dots$ となっていることを確かめてください.

▶ 解答

5.4. 実装例

Typical DP Contest 「T - フィボナッチ」 への提出です.

- <https://atcoder.jp/contests/tdpc/submissions/76156813>

この実装では, Bostan-Mori のアルゴリズムを用いています. 多項式乗算は素朴な方法により計算しているため, 漸化式の位数を d , 求める添字を K とすると, 解法全体の計算量は $O(d^2 \log K)$ 時間です (問題文の記号では, $O(K^2 \log N)$ 時間です).

もうひとつの実装例を挙げます. これは Library Checker 「Kth term of Linearly Recurrent Sequence」 への提出です. 多項式乗算を FFT による畳み込みで行うことで, 計算量は $O(d \log d \log K)$ 時間になります. 畳み込みの実装には AtCoder Library を用いています.

- <https://judge.yosupo.jp/submission/375054>

6. 関連問題

関連問題として挙げた問題は, 一部 線形漸化的数列 と重複しています.

1. https://atcoder.jp/contests/tdpc/tasks/tdpc_fibonacci
2. <https://yukicoder.me/problems/no/891>
3. https://atcoder.jp/contests/math-and-algorithm/tasks/math_and_algorithm_av

4. https://atcoder.jp/contests/abc009/tasks/abc009_4
5. https://judge.yosupo.jp/problem/kth_term_of_linearly_recurrent_sequence
6. <https://yukicoder.me/problems/no/215>
7. https://atcoder.jp/contests/abc178/tasks/abc178_d
8. https://atcoder.jp/contests/s8pc-3/tasks/s8pc_3_g
9. https://atcoder.jp/contests/qjo2025-final/tasks/qjo2025_final_b
10. https://atcoder.jp/contests/abc436/tasks/abc436_g
11. https://atcoder.jp/contests/abc300/tasks/abc300_h
12. https://atcoder.jp/contests/fps-24/tasks/fps_24_t

▶ 問題について

7. まとめ

本講座では、定数係数線形漸化式と初期値が与えられたときに、数列の第 K 項を高速に求める方法として、Fiduccia のアルゴリズムと Bostan–Mori のアルゴリズムを解説しました。

Fiduccia のアルゴリズムは、線形漸化的数列の第 K 項計算を、特性多項式 $\Gamma(x)$ による剰余 $x^K \bmod \Gamma(x)$ の計算と結びつける方法です。この考え方は、線形漸化的数列と、同伴行列・行列累乗・行列の特性多項式などの関係を理解する上でも重要です。

一方、Bostan–Mori のアルゴリズムは、線形漸化的数列の第 K 項計算を母関数の係数 $[x^K] \frac{P(x)}{Q(x)}$ と考えて計算する方法です。

どちらの方法も、多項式乗算の計算量を $M(d)$ とすると、 $O(M(d) \log K)$ 時間で第 K 項を求めることができます。特に、競技プログラミングで頻出する $R = \mathbb{Z}/n\mathbb{Z}$ における計算では $M(d) = O(d \log d)$ と考えることができ、計算量は $O(d \log d \log K)$ 時間となります。

本講座では計算量をオーダーの意味で評価しましたが、算術計算量、すなわち係数環 R における環演算の回数をより詳しく見ると、定数倍にも違いがあります。標準的な高速多項式演算を用いる場合、おおまかには

- Fiduccia のアルゴリズム : $3M(d) \log K$ 程度
- Bostan–Mori のアルゴリズム : $2M(d) \log K$ 程度

と評価でき、この意味では Bostan-Mori のアルゴリズムの方が一般に高速です。さらに FFT を用いる場合には、両者とも定数倍改善の余地がありますが、Bostan-Mori のアルゴリズムでは主要項を $\frac{2}{3}M(d) \log K$ 程度まで小さくできることが知られています。

このような定数倍高速化のテクニックや、本講座で扱わなかった Bostan-Mori のアルゴリズムの発展的な話題については以下の講座で扱います。

- Bostan-Mori のアルゴリズム（発展）

また、本講座では線形漸化式あるいは母関数の有理式表示が与えられている数列を扱いました。ここで扱った内容に加えて、線形漸化式の復元について学ぶことで、本講座の手法がさらに競技プログラミングの問題に適用しやすくなります。線形漸化式の復元については以下の講座で扱います。

- 線形漸化式の復元
- 線形漸化式の復元 (mod m)

8. 参考文献

1. Charles M. Fiduccia. An Efficient Formula for Linear Recurrences. SIAM Journal on Computing. Vol. 14. No. 1. pp. 106–112. 1985. DOI: 10.1137/0214007. <https://doi.org/10.1137/0214007>.
2. Alin Bostan, Ryuhei Mori. A Simple and Fast Algorithm for Computing the N-th Term of a Linearly Recurrent Sequence. Proceedings of Symposium on Simplicity in Algorithms (SOSA). pp. 118–132. 2021. DOI: 10.1137/1.9781611976496.14. <https://doi.org/10.1137/1.9781611976496.14>. (Preprint). <https://arxiv.org/abs/2008.08822>.
3. Ryuhei Mori. 線形漸化式的数列のN項目の計算. Qiita. <https://qiita.com/ryuhe1/items/da5acbcce4ac1911f47a>. (閲覧日: 2026-05-27).
4. Misuki. [tutorial] Bostan-Mori, an elegant algorithm to compute k-th term of linear recurrence in $O(d \lg d \lg k)$. Codeforces Blog. <https://codeforces.com/blog/entry/111862>. (閲覧日: 2026-05-27).
5. OI Wiki. 常系数齐次线性递推. <https://oi-wiki.org/math/poly/linear-recurrence/>. (閲覧日: 2026-05-27).

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

